Building a Simulator for a Sparse Accelerator

CS6886 (System Engineering for Deep Learning) – Assignment 4

Department of Computer Science and Engineering, IIT Madras

Due date: 14 October 2026, 11:59 PM

1. Overview

You will complete a simulator of a SparTen-style sparse CNN accelerator (Gondimalla et al., *SparTen: A Sparse Tensor Accelerator for Convolutional Neural Networks*, MICRO 2019) and use it to run a real pruned INT8 CNN on the MNIST handwritten digit recognition dataset. You will lower each layer to a matrix multiplication, tile it to fit 16 processing elements (PEs), implement two loop orders, profile the result, and report the following inference metrics: cycles, utilization, energy, power, and the roofline. The two loop orders are the inner product, which SparTen uses, and the outer product, which SCNN uses (Parashar et al., *SCNN: An Accelerator for Compressed-sparse Convolutional Neural Networks*, ISCA 2017). SCNN is the earlier sparse accelerator that SparTen is compared against.

By the end of this assignment, you should be able to:

- lower convolution and fully-connected layers to a matrix multiplication (GEMM);
- map a GEMM onto a fixed array of PEs with tiling and a loop order, and compare the inner and outer product;
- build a simulator that counts events (MACs, cycles, bytes) and turns them into latency, energy, power, and roofline metrics;
- read a roofline and decide what limits each layer;
- explain how much of a dense CNN's work sparsity eliminates, and why the real speedup is smaller.

2. What you are given

- **sparten.c** – the top-level simulator. You complete the 8 TODOs in this file. Everything else in it is given: the batch machinery, the network driver, the correctness checks, and the post-processing steps (bias, requantization, ReLU, max-pool).
- **pe.h** – the PE-level simulator (given, do not change). It models one SparTen PE (the paper calls it a compute unit): the SparseMap format, the priority encoder, the prefix sums, `pe_inner` and `pe_outer`.
- **mnist_model.h** – the pruned INT8 network: conv1 (1 to 8 channels, 3×3), 2×2 max-pool, conv2 (8 to 16 channels, 3×3), 2×2 max-pool, fc1 (784 to 32), fc2 (32 to 10). It reaches 98.25% accuracy on the 10,000 MNIST test images.
- **mnist_images.h** – 100 test images, their labels, and the digit the reference model predicts for each.
- **README.md** – a short summary of the files and the TODOs.

3. SparTen Model

Processing Element (pe.h)

A PE works on vectors of up to 16 values. Each vector is stored as a 16-bit SparseMap (bit i is 1 if element i is non-zero) plus its non-zero values packed in order; `encode()` builds this format. `pe_inner` computes the dot product of two vectors the way a SparTen PE does:

1. the priority encoder holds `mask_a AND mask_b`; every cycle it issues the lowest set position (a match) and clears that bit;
2. two prefix sums count the 1s below that position in each mask, which is where each operand's value sits in its packed array;
3. the accumulator reads both values, multiplies them, and accumulates the product.

`pe_outer` multiplies every non-zero of one vector with every non-zero of the other. It walks the set bits of the two masks in order, so it needs no AND, because every pair of non-zeros gives a useful product, and no prefix sums, because the packed values are read in order.

PE Timing Information

The work a PE does on one pair of vectors is called a *chunk*. We measured the PE's timing on the RTL of our own PE implementation. The PE handles one match per cycle, and filling and draining its pipeline adds 8 cycles, so a chunk with m matches takes $m + 8$ **cycles**. A chunk with no matches takes **7 cycles**. The outer product is assumed to use the same pipeline, with one product per cycle in place of one match. The clock frequency is 800 MHz.

Energy

Table 4 of the SparTen paper reports that one 32-PE cluster, synthesized in 45 nm technology and clocked at 800 MHz, consumes 118.3 mW of power in total. `pe.h` converts each component's power into an energy per match, in picojoules (pJ).

Memory

The PEs read their operands from on-chip SRAM through a 128-bit port: 16 bytes per cycle. Every vector, in both loop orders, is stored in the SparseMap format and costs 2 bytes of SparseMap plus 1 byte per non-zero value, even when it holds fewer than 16 values. A partial sum costs 4 bytes.

Layers as matrix multiplications

Every layer becomes `C[rows][cols] = A[rows][depth] x B[depth][cols]`:

Layer	rows	depth	cols	
conv1	784	9	8	rows = 28×28 positions, depth = $3 \times 3 \times 1$
conv2	196	72	16	rows = 14×14 positions, depth = $3 \times 3 \times 8$
fc1	1	784	32	one image, depth = $7 \times 7 \times 16$ inputs
fc2	1	32	10	

The multiplication is cut into tiles that fit the 16 PEs (Figure 1). C is divided into *output tiles* of `TILE_ROWS x TILE_COLS` outputs, and the depth into *depth tiles* of 16, the PE's vector length. One tile is one output tile combined with one depth tile: it needs a `TILE_ROWS x 16` block of A (the *A block*) and a `16 x TILE_COLS` block of B (the *B block*). `TILE_ROWS x TILE_COLS` must equal 16; the default tile is $4 \times 16 \times 4$.

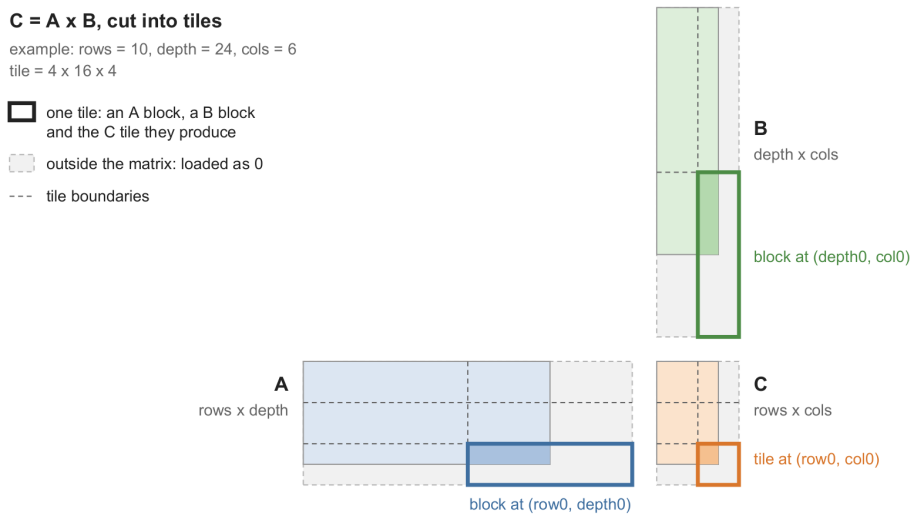


Figure 1. One tile is a `TILE_ROWS x 16` block of A, a `16 x TILE_COLS` block of B, and the `TILE_ROWS x TILE_COLS` tile of C that they produce. The tile shown is the last one in every dimension, so all three blocks stick out of their matrices. The part outside a matrix is loaded as 0.

Batches

Each tile runs as one *batch* of up to 16 PE tasks, where a task is one PE processing one chunk. The next batch starts only when all 16 PEs are done; there is no load balancing. The *compute cycles* of a batch are therefore those of its slowest PE. In the outer product, the products for one output come from several PEs, and the accumulator of each of the 16 outputs can add only one value per cycle, so a batch cannot finish before its busiest accumulator either.

The *memory cycles* of a batch are the bytes it moves divided by the bandwidth (16 bytes per cycle by default, set by `BYTES_PER_CYCLE`), rounded up. The operands for a batch are transferred while the PEs compute, so the two overlap, and a batch takes whichever is larger: its compute cycles or its memory cycles.

Roofline

Peak compute is 16 multiply-accumulate operations (MACs) per cycle, which is 32 operations per cycle, since a MAC counts as two operations (a multiply and an add). Peak bandwidth is 16 bytes per cycle. The ridge point is therefore $32/16 = 2$ operations per byte.

4. Building and running

```
gcc -O2 -Wall sparten.c -o sparten -lm && ./sparten
gcc -O2 -Wall -DTILE_ROWS=2 -DTILE_COLS=8 sparten.c -o sparten -lm   # another tile
gcc -O2 -Wall -DBYTES_PER_CYCLE=8 sparten.c -o sparten -lm           # less bandwidth
gcc -O2 -Wall -DNUM_IMAGES=5 sparten.c -o sparten -lm                # fewer images
```

The program runs all 100 images through the network, running each layer twice (once with the inner product and once with the outer product). It prints the accuracy, a per-layer table (cycles, utilization, energy, power) and a roofline table, and writes `roofline.csv` and `roofline.svg` (open the SVG in a web browser).

You can compile and run the program before completing any TODO. It will run, but it prints CHECK lines, a low accuracy, and zeros in the tables; this is expected. A CHECK line means that one of the correctness checks failed, and it names the TODO to look at, for example `CHECK conv1: the lowered GEMM does not compute the layer (TODO 1/2)`. The CHECK lines disappear and the tables fill in as you complete the TODOs. Only the inner product's result is passed on to the next layer, so the accuracy depends on TODOs 1–4; the outer product is checked only by its own CHECK line.

The last line of the output is a sanity check. It prints a WARNING line for every impossible result, such as a utilization above 100%, a performance above the roofline, or fewer bytes than are needed to write each output once, and names the TODO to look at. The absence of warnings does not mean that your numbers are right, only that they are possible. There is no reference output to compare with, so check your numbers against the worked examples in TODO 7 and by hand in experiment E4.

5. The TODOs

TODO 1 – lower_fc: a fully-connected layer as a GEMM

```
void lower_fc(const Layer *L, const Tensor *in, Matrix *A, Matrix *B)
```

- A has a single row, with one column per input value. B has one row per input value and one column per output. Element (d, o) of B is the weight that connects input d to output o.
- The input tensor is stored channel-last; flatten it in that same order.
- The weights are stored output by output: all the weights of output 0, then all the weights of output 1, and so on.
- Checked by: A $\times$ B must equal the fully-connected layer computed directly.

TODO 2 – lower_conv: a 3×3 convolution as a GEMM (im2col)

```
void lower_conv(const Layer *L, const Tensor *in, Matrix *A, Matrix *B)
```

- A has one row per output position, taken row by row (stride 1 and padding 1, so the output has the same height and width as the input), and one column per element of the 3×3 window across all input channels.
- B has one row per window element and one column per filter.
- Order the window elements the way the filter's weights are stored: by filter row, then filter column, and then channel.
- Window elements that fall outside the image are set to 0 (padding).
- Checked by: A $\times$ B must equal the convolution computed directly.

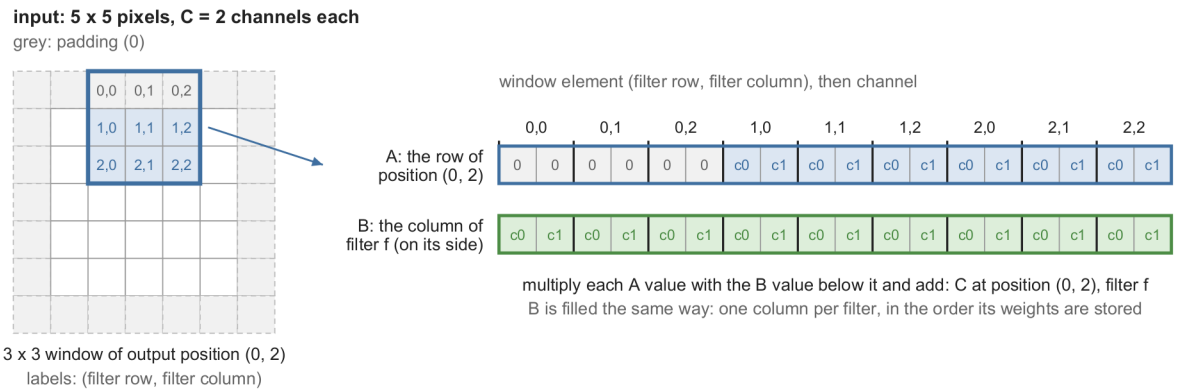


Figure 2. im2col for one output position, with 2 input channels. The 3×3 window becomes one row of A, element by element, and each element contributes all its channels. The window of position (0, 2) sticks out above the image, so its first three elements are 0.

TODO 3 – load_tile: cut one tile out of A and B

```
void load_tile(const Matrix *A, const Matrix *B, int row0, int depth0, int col0,
               int a[TILE_ROWS][TILE_DEPTH], int b[TILE_DEPTH][TILE_COLS])
```

- Copy into `a` the `TILE_ROWS x 16` block of A that starts at row `row0`, column `depth0`, and into `b` the `16 x TILE_COLS` block of B that starts at row `depth0`, column `col0`.
- Any part of a block that falls outside its matrix is set to 0, since the last tile in each dimension can be partial.
- Checked by: the results of TODOs 4 and 5.

TODO 4 – gemm_inner: the inner-product loop order

```
void gemm_inner(const Matrix *A, const Matrix *B, Matrix *C)
```

- Every output tile of C is computed by going through its depth tiles in order. Each depth tile is one batch, between `start_batch()` and `end_batch()`.
- Within a batch, each of the 16 PEs computes one output of the tile (Figure 3, left): PE (r, c) gets row r of the A block and column c of the B block (16 values each, stored with `encode()`) and adds the result of `pe_task_inner()` to that output's partial sum.
- Every PE whose output lies inside C gets a task, even if both of its vectors are all zeros. A PE whose output lies outside C gets no task.
- When the last depth tile is done, write the tile's outputs into C.
- Checked by: C must equal a plain matrix multiply.

Each PE owns one output and keeps its partial sum while the depth tiles stream past. This is the output-stationary dataflow that SparTen uses.

TODO 5 – gemm_outer: the outer-product loop order

```
void gemm_outer(const Matrix *A, const Matrix *B, Matrix *C)
```

- The depth tile is the outermost loop: for each depth tile, every output tile of C is one batch, between `start_batch()` and `end_batch()`.
- Within a batch, each PE handles one depth index k of the tile (Figure 3, right): it gets column k of the A block (`TILE_ROWS` values) and row k of the B block (`TILE_COLS` values), stored with `encode()`, and calls `pe_task_outer()`, which adds its products into the tile's partial sums.
- Every PE whose depth index lies inside the depth of A gets a task, even if both of its vectors are all zeros. A PE whose depth index lies past it gets no task.
- After each batch, add the tile's partial sums into C, which starts at zero.
- Checked by: C must equal a plain matrix multiply.

This is the dataflow of SCNN (Section 1). It needs no matching, because every non-zero in column k of the A block is multiplied with every non-zero in row k of the B block. The cost is that the products for one output come from different PEs, so the partial sums have to be merged: across the PEs within a batch, and through C across depth tiles. SCNN itself stores its data in a compressed format with explicit indices; here both loop orders use the SparseMap, so the comparison isolates the effect of the loop order.

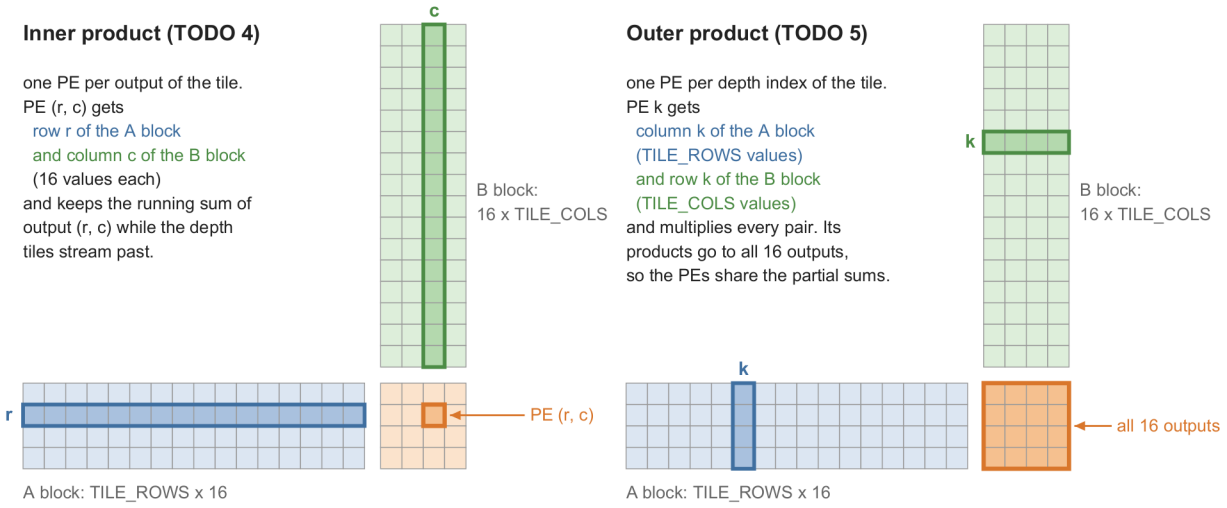


Figure 3. Left, inner product: each PE owns one output and matches a row of the A block against a column of the B block. Right, outer product: each PE owns one depth index and multiplies a column of the A block by a row of the B block, adding into the partial sums of the whole tile.

TODO 6 – layer_metrics: profiling numbers per image

```
Metrics layer_metrics(const Stats *s, int num_images)
```

The `Stats` hold totals over `num_images` images, and record energy in picojoules (pJ). Every metric you return is per image, and energy is returned in nanojoules (1 nJ = 1000 pJ). A *useful* MAC is one whose two operands are both non-zero.

- **cycles** – cycles per image;
- **time_us** – time per image in microseconds, at `CLOCK_MHZ`;
- **utilization** – the fraction of all PE-cycles (16 PEs $\times$ cycles) spent on a useful MAC;
- **macs_per_cycle** – useful MACs per cycle;
- **energy_nj** – energy per image in nanojoules, converted from the picojoules in the `Stats`;
- **power_mw** – average power in milliwatts while the layer runs.

Return all zeros if there are no cycles. Checked by: the WARNING lines of the sanity check.

TODO 7 – count the bytes each batch moves (inside gemm_inner and gemm_outer)

For every batch, record the bytes it moves by calling the given `count_bytes(n)` before `end_batch()`. The argument `n` is a number of bytes moved between SRAM and the PEs. Each call adds `n` to the current batch's total, so you can call it once per vector or once with the sum, and `end_batch()` uses the total to compute the batch's memory cycles. Use `vector_bytes(&v)` for the size of a stored vector `v` (a vector with no non-zeros still costs the 2 bytes of its SparseMap) and `PSUM_BYTES` (4) for a partial sum.

- **Inner product:** each row of the A block and each column of the B block is read once and broadcast to all the PEs that use it. Count each one once, not once per PE, and count only the rows and columns that belong to outputs inside C. In the last depth tile, also count `PSUM_BYTES` for each finished output written to C.
- **Outer product:** every PE reads its own column of the A block and row of the B block, so count both for every PE that has a task. After every depth tile, count `PSUM_BYTES` written for each output of the tile that lies inside C. From the second depth tile on, also count `PSUM_BYTES` read for each of those outputs.
- Checked by: the worked examples below and the WARNING lines of the sanity check.

Worked example, inner product. Consider a $4 \times 16 \times 4$ tile at the bottom edge of C: only 2 of its rows lie inside C, all 4 of its columns do, and this is the last depth tile (Figure 4).

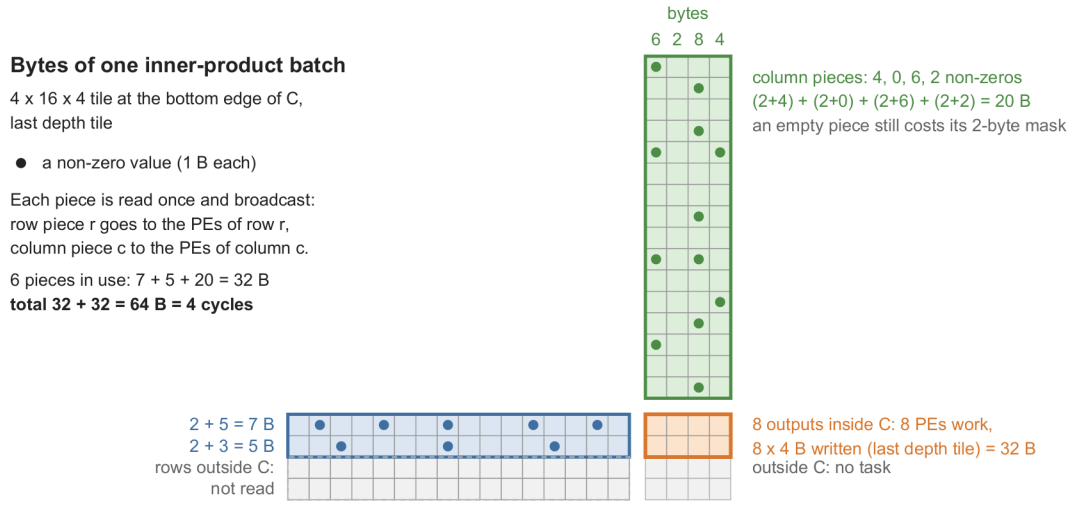


Figure 4. The inner-product worked example. Each row of the A block and each column of the B block is stored as a 2-byte SparseMap plus 1 byte per non-zero, and is read once however many PEs use it.

rows of A in use	2, with 5 and 3 non-zeros	(2+5)+(2+3)	= 12 bytes
columns of B	4, with 4, 0, 6, 2 non-zeros	(2+4)+(2+0)+(2+6)+(2+2)	= 20 bytes
outputs written	2×4 outputs, last depth tile	8×4	= 32 bytes
		total	= 64 bytes
memory time	64 / 16 bytes per cycle		= 4 cycles

Only the 8 PEs whose outputs lie inside C (2 rows $\times$ 4 columns) have a task. Between them they use 6 vectors: 2 rows of the A block and 4 columns of the B block. Each vector is read once and broadcast, so it is counted once. Counting a vector once for every PE that uses it would give 16 vectors (8 PEs $\times$ 2 vectors each), which is wrong.

Worked example, outer product. Suppose A has a depth of 26. Its second depth tile (`depth0 = 16`) then holds 10 depth indices, so 10 PEs get a task. All 16 outputs of the tile lie inside C. The 10 columns of the A block hold 17 non-zeros in total, and the 10 rows of the B block hold 23.

columns of A	$10 \times 2 + 17$	= 37 bytes
rows of B	$10 \times 2 + 23$	= 43 bytes
partial sums	(16 written + 16 read, not the first depth tile) $\times$ 4	= 128 bytes
	total	= 208 bytes
memory time	208 / 16 bytes per cycle	= 13 cycles

TODO 8 – roofline_point: place a layer on the roofline

```
RooflinePoint roofline_point(const Stats *s)
```

Count a MAC as two operations (a multiply and an add). The `Stats` hold totals over all images. The roofline has two roofs. The **memory roof** is the performance that the bandwidth allows at a given intensity, `BYTES_PER_CYCLE` $\times$ intensity. The **compute roof** is the peak performance, `PEAK_OPS_PER_CYCLE`. Return:

- **intensity** – operations per byte moved;
- **achieved** – operations per cycle reached;
- **attainable** – the lower of the two roofs at this intensity;
- **memory_bound** – 1 when the memory roof is the lower one (the bandwidth is the limit), otherwise 0.

Return all zeros if there are no bytes or no cycles. Checked by: the WARNING lines of the sanity check.

Suggested order

1, 2 (the lowering checks pass), 3, 4 (the inner-product check passes and accuracy reaches 100/100), 5 (the outer-product check passes), 6, 7, 8 (no WARNING lines, and your byte counts agree with the worked examples).

6. Experiments

Once all TODOs are done, run these and keep the output for your report.

- **E1 – baseline:** the default run.
- **E2 – tile shape:** `TILE_ROWS x TILE_COLS` = 1×16, 2×8, 4×4, 8×2 and 16×1.
- **E3 – bandwidth:** `BYTES_PER_CYCLE` = 4, 8, 16, 32 and 64 with the default tile.
- **E4 – one batch by hand:** for image 0, conv1, the first batch of `gemm_inner` (`row0 = col0 = depth0 = 0`), print the number of matches of each of the 16 PE tasks: it is the number of 1 bits in the AND of the two vectors' masks (add a printf in your code). Compute by hand the batch's compute cycles, its bytes and its memory cycles, and check them against what the simulator reports.

7. Report

Two pages of text, plus at most two pages for figures and tables (the roofline plot and the tile-shape and bandwidth sweeps). Answer each question with numbers from your own runs.

1. **Validation.** How do you know your simulator is right? Give your hand calculation of the batch in E4 and compare it with the simulator.
2. **Where the cycles go.** 16 PEs doing one useful MAC per cycle would need sparse MACs / 16 cycles. Compare that with the measured cycles for each layer and split the gap into its causes: the pipeline fill and drain of every chunk, waiting for the slowest PE, PEs left idle in partial tiles, and memory stalls.
3. **Inner versus outer product.** Which loop order is faster for each layer, and why? Use the batches, bytes, and accumulator limit in your explanation (look at conv1 and fc1).
4. **Roofline.** Include the roofline plot. Every layer sits left of the ridge, yet the inner product reaches only part of its roof while having almost no memory-bound batches. Explain what actually limits each layer.
5. **Tile shape.** From E2: how does the tile shape change reuse (bytes) and idle PEs? Which tile is best for each layer, and how much faster would the network be with the best tile per layer?
6. **Bandwidth.** From E3: at what bandwidth does each dataflow stop speeding up, and why?
7. **Energy.** Which dataflow uses less energy per image, and where does the difference come from?
8. **One improvement.** Propose one change to the hardware or the mapping (for example, longer chunks, letting PEs run ahead without a barrier, load balancing, or batching images in the fully-connected layers). Estimate its effect with numbers from your simulator.

8. Submission

- Submit one zip with your `sparten.c` and `report.pdf`.
- Do not change `pe.h`, the model and image headers, or the given parts of `sparten.c`: your code is graded with our copies of those files, with other tile shapes and bandwidths.
- Your program must compile without warnings with the command in Section 4.

9. Grading (100 marks)

Code – 60 marks

Each TODO function is tested automatically and on its own against the reference implementation, on the default settings and on other tile shapes (`TILE_ROWS x TILE_COLS`) and bandwidths (`BYTES_PER_CYCLE`). Marks are given in proportion to the test cases passed. Your TODO functions are tested inside our copy of the given code, so changes outside them have no effect. You may add your own helper functions.

TODO	Function	Marks
1	`lower_fc`	5
2	`lower_conv`	10
3	`load_tile`	5
4	`gemm_inner`	10
5	`gemm_outer`	10
6	`layer_metrics`	5
7	byte counting	10
8	`roofline_point`	5

A program that does not compile with the command in Section 4 gets 0 code marks. One that compiles but prints warnings gets at most half of the code marks.

Report – 40 marks

Question	Topic	Marks
Q1	validation (hand-computed batch)	5
Q2	where the cycles go	6
Q3	inner versus outer product	5
Q4	roofline	6
Q5	tile shape	6
Q6	bandwidth	4
Q7	energy	3
Q8	one improvement, with an estimate	5

Each answer is marked on whether it uses numbers from your own runs and whether it explains the cause, not only the result.